

Systèmes Distribués et Intelligence Artificielle —
Technologies Web

Rapport de Projet

— Mourchid : Assistant académique

ENSET
multi-agent avec RAG agentique

RÉALISÉ PAR :

Ahmed DOUYRY

Yazid ESSOKTANI

Imane MEKKAOU

 [github.com/ ahmed-douyry/ enset-
assistant](https://github.com/ahmed-douyry/enset-assistant)

ENCADRÉ PAR :

Oumaima AGHERAI

Fait le : Juin 2026

TABLE DES MATIÈRES

1	Introduction	2
1.1	Présentation du projet	2
1.2	Objectifs du projet	2
2	Cahier des charges	2
2.1	Problématique	3
2.2	Besoins identifiés	3
2.3	Solutions proposées	4
3	Conception fonctionnelle	4
3.1	Identification des acteurs	4
3.2	Diagramme de cas d'utilisation	5
3.3	Diagramme de classes	6
3.4	Diagramme de séquence — flux de chat (streaming SSE)	7
3.5	Diagramme de séquence — dictée vocale (Whisper)	8
4	Conception technique	8
4.1	Architecture générale	8
4.2	Stack technologique	9
4.3	Structure du dépôt	9
4.4	Graphe LangGraph	9
4.5	Endpoints REST principaux	10
4.6	Composant de dictée vocale	11
4.7	Génération de quiz par le LLM	11
5	Réalisation	11
5.1	Interface assistant académique	12
5.2	Dictée vocale avec barre de fréquence	12
5.3	Quiz concours généré par l'IA	13
5.4	Gestion documentaire	13
5.5	Visualisation du workflow multi-agent	14
5.6	Tableau de bord	14
5.7	Résumé structuré de documents	15
6	Installation et déploiement	15
7	Conclusion	16
7.1	Résumé du projet	16
7.2	Défis rencontrés	17
7.3	Recommandations et perspectives	17

1. Introduction

Contexte académique

Ce projet est réalisé dans le cadre du module **Systèmes Distribués et Intelligence Artificielle — Technologies Web** du **Master SDIA** à l'**ENSET Mohammedia** (Université Hassan II de Casablanca). Il vise à concevoir et implémenter une application web complète intégrant des composants d'intelligence artificielle distribués.

1.1 Présentation du projet

Mourchid est un assistant académique intelligent et **local-first** dédié aux étudiants et futurs candidats de l'**ENSET Mohammedia**. Il permet de rechercher, comprendre et structurer des informations relatives à la vie académique de l'école : filières du cycle ingénieur, procédures d'inscription, bourses, stages/PFE, clubs étudiants et concours d'accès.

« *Mourchid* » (مورخيد) signifie *guide en arabe* — métaphore choisie pour exprimer la vocation de l'assistant : orienter l'étudiant dans ses démarches académiques à partir de sources documentaires vérifiées.

1.2 Objectifs du projet

- Fournir un **assistant conversationnel intelligent** capable de répondre aux questions académiques en se basant sur un corpus de documents officiels ENSET.
- Implémenter un **pipeline RAG agentique** (Retrieval-Augmented Generation) multi-agent orchestré par **LangGraph**.
- Intégrer des fonctionnalités innovantes : **dictée vocale** avec transcription locale (Whisper), **quiz concours** générés par l'IA, visualisation du workflow multi-agent.
- Assurer une architecture **100% locale et open-source**, sans dépendance à des APIs cloud propriétaires.
- Respecter les bonnes pratiques **Technologies Web** : API REST, streaming SSE, interface React responsive.



Avertissement

Les documents fournis dans le corpus sont des **échantillons pédagogiques** à titre de démonstration. L'assistant ne remplace pas le service de scolarité de l'ENSET ni les communications officielles de l'UH2C. Pour toute décision administrative, se référer aux services compétents.

2. Cahier des charges

2.1 Problématique

Les étudiants souhaitant intégrer l'ENSET Mohammedia ou déjà inscrits font face à plusieurs difficultés récurrentes :

- **Dispersion de l'information** : les renseignements sur les filières, les procédures administratives (inscription, bourse, attestations), les clubs et les emplois du temps sont éparpillés sur plusieurs supports.
- **Manque de disponibilité** : le service de scolarité n'est pas accessible en dehors des heures de bureau.
- **Préparation insuffisante au concours** : les candidats manquent d'outils interactifs pour s'entraîner aux QCM par filière.
- **Barrière de langue et de format** : les documents officiels sont souvent denses et difficiles à parcourir.

2.2 Besoins identifiés

Catégorie	Besoins
Fonctionnels	Répondre aux questions académiques en langage naturel, proposer des procédures étape par étape, générer des résumés de documents, entraîner aux quiz concours.
Accessibilité	Interface disponible 24h/24, saisie par clavier ou voix (dictée vocale).
Transparence	Citer les sources utilisées, afficher le score de confiance, permettre l'audit du pipeline agent.
Performance	Réponses en streaming (pas d'attente bloquante), interface réactive.
Souveraineté	Fonctionnement 100% local, aucune donnée envoyée à des serveurs tiers.

TABLE 1 – Synthèse des besoins identifiés.

2.3 Solutions proposées

Problème	Solution retenue	Statut
Information dispersée	Pipeline RAG (Qdrant + SentenceTransformers) sur un corpus ENSET unifié	✓
Disponibilité	Application web accessible en permanence sur localhost:5173	✓
Saisie difficile	Bouton micro + transcription Whisper locale côté serveur + barre de fréquence	✓
Préparation concours	Quiz QCM générés par le LLM à chaque essai (par filière, par difficulté)	✓
Réponses opaques	Score de confiance, citations RAG, panneau workflow, traces persistées	✓
Dépendance cloud	Stack 100% locale : Ollama, Qdrant, Whisper, SentenceTransformers	✓

TABLE 2 – Cartographie problèmes ↔ solutions.

3. Conception fonctionnelle

3.1 Identification des acteurs

Acteur	Rôle
Étudiant / Candidat	Interroge l'assistant, s'entraîne aux quiz, utilise la dictée vocale et consulte les résumés.
Administrateur	Charge et indexe des documents dans le corpus via l'interface de gestion.
Système Lang-Graph	Orchestre les agents (classification, RAG, raisonnement, citations, vérification).
LLM Ollama	Génère les réponses, classe les questions, génère les quiz.
Whisper (Transformers)	Transcrit l'audio capturé par le micro en texte.
Qdrant Chroma	/ Stocke et recherche les vecteurs du corpus documentaire.

TABLE 3 – Acteurs du système Mourchid.

3.2 Diagramme de cas d'utilisation

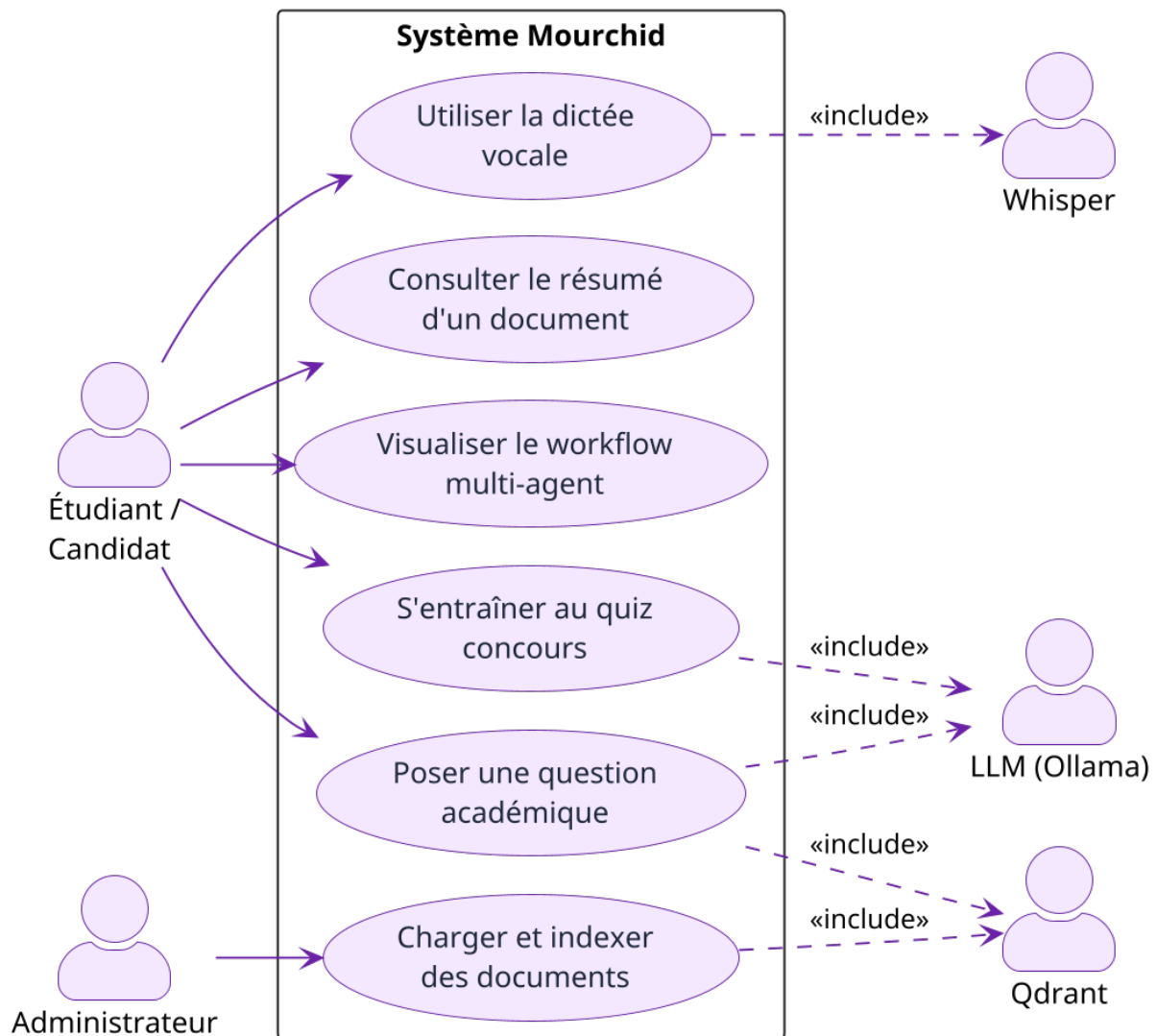


FIGURE 1 – Diagramme de cas d'utilisation — système Mourchid (généré avec PlantUML).

3.3 Diagramme de classes

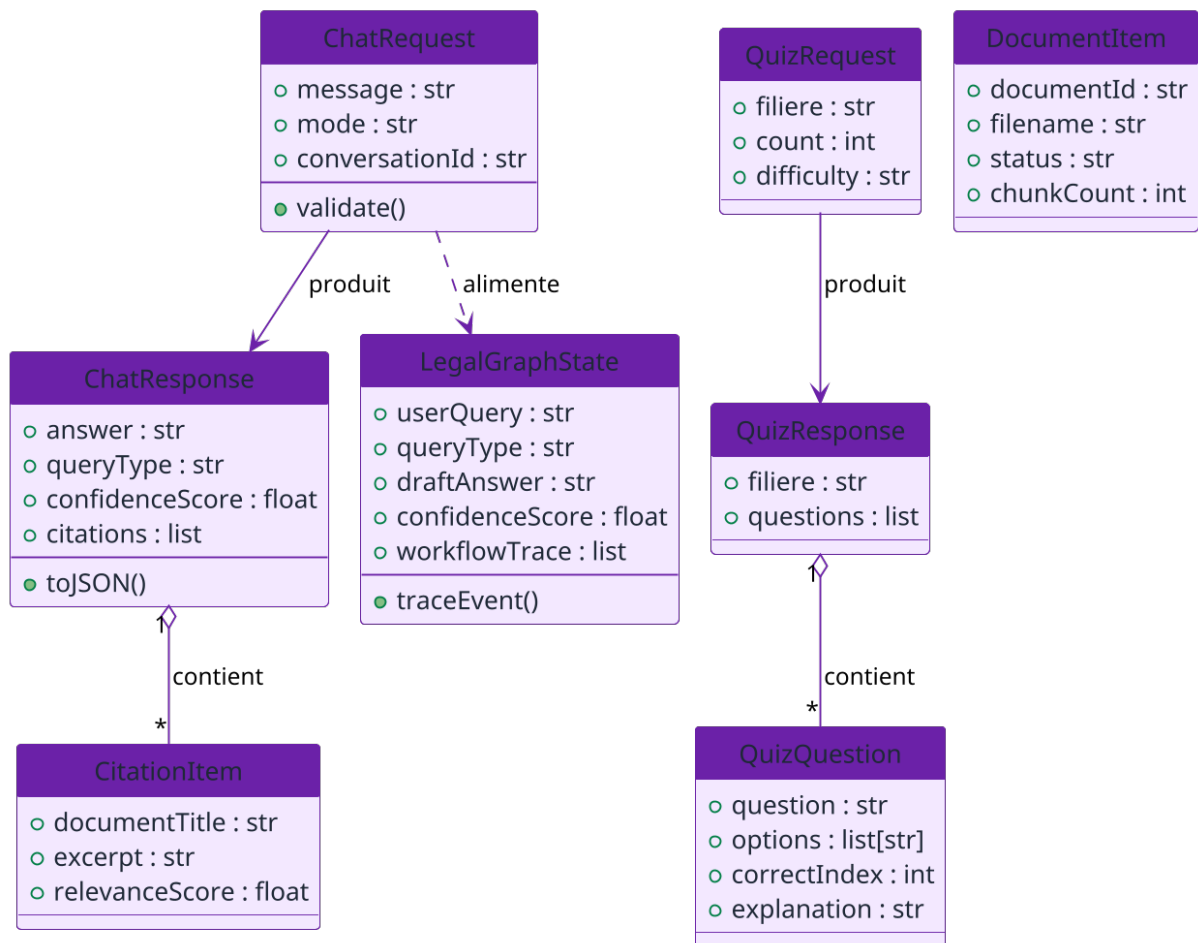


FIGURE 2 – Diagramme de classes — modèles de données de l'API Mourchid (généré avec PlantUML).

3.4 Diagramme de séquence — flux de chat (streaming SSE)

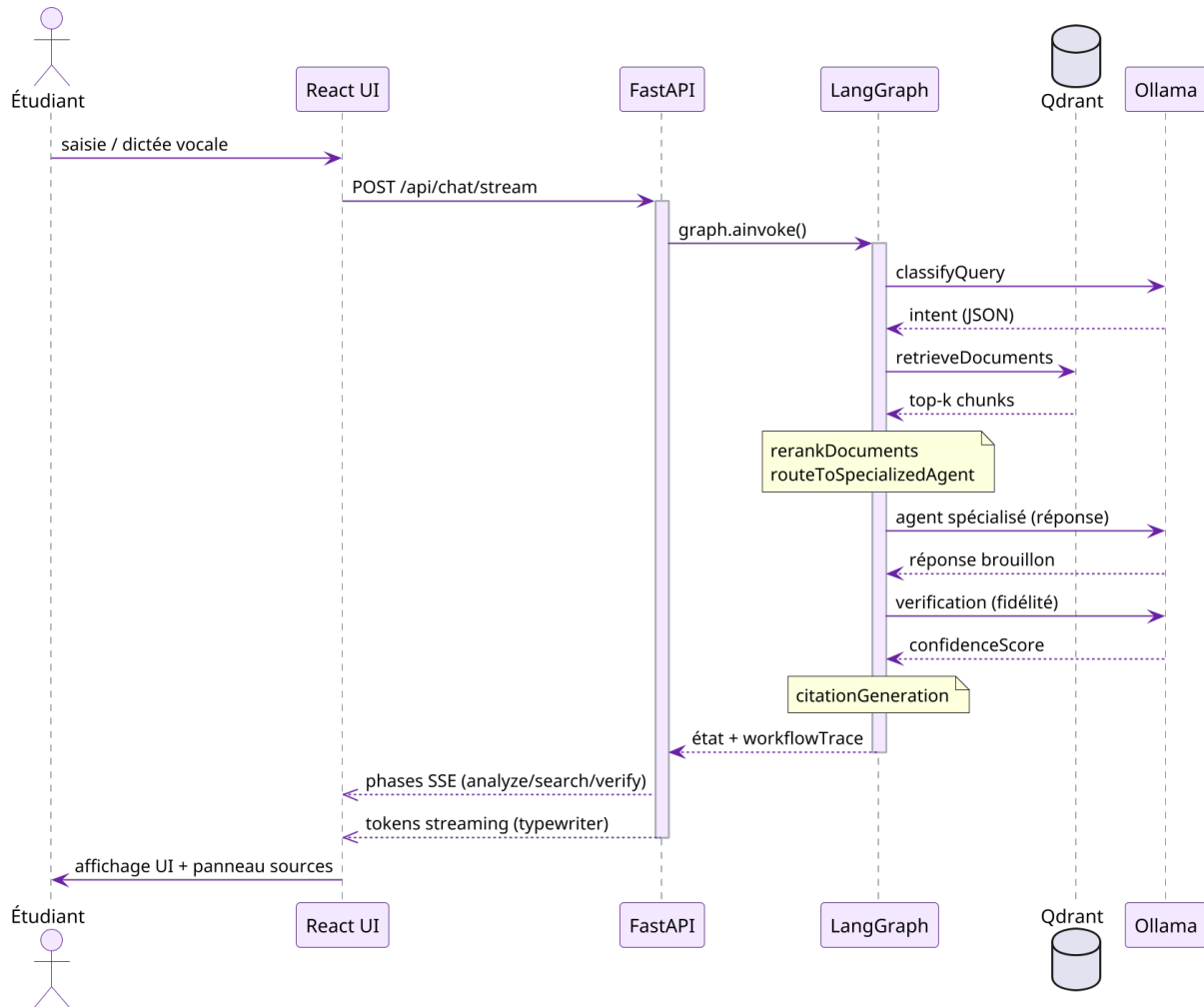


FIGURE 3 – Diagramme de séquence UML — parcours chat avec streaming SSE multi-agent (généré avec PlantUML).

3.5 Diagramme de séquence — dictée vocale (Whisper)

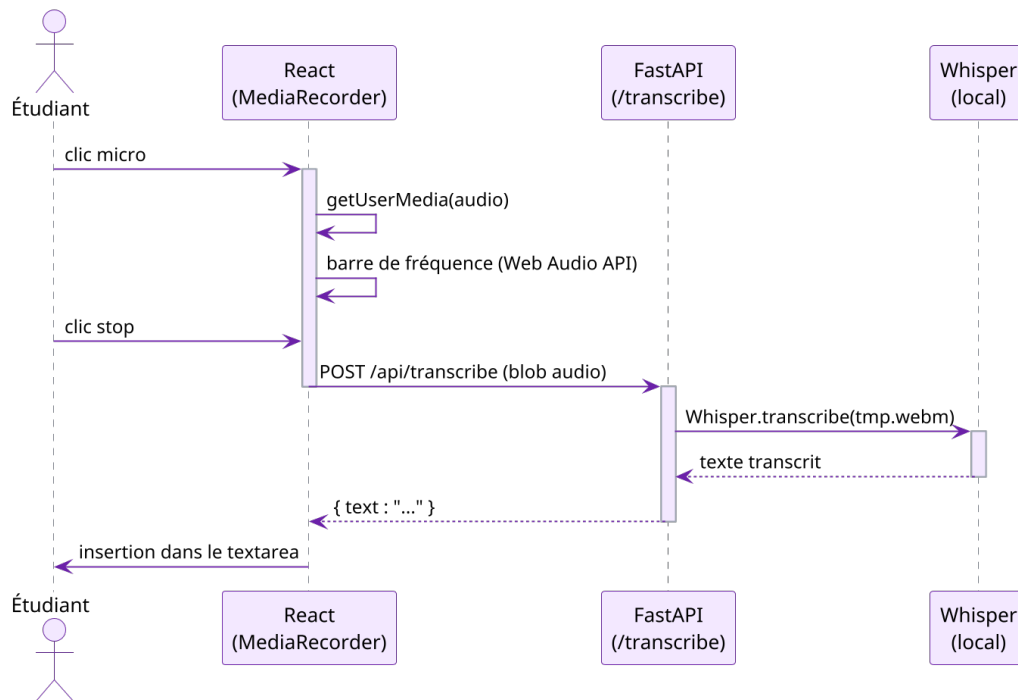


FIGURE 4 – Diagramme de séquence — dictée vocale et transcription Whisper côté serveur (généré avec PlantUML).

4. Conception technique

4.1 Architecture générale

L'application suit une architecture **monorepo** séparant clairement le backend Python et le frontend TypeScript, communiquant via **API REST** et **Server-Sent Events (SSE)**.

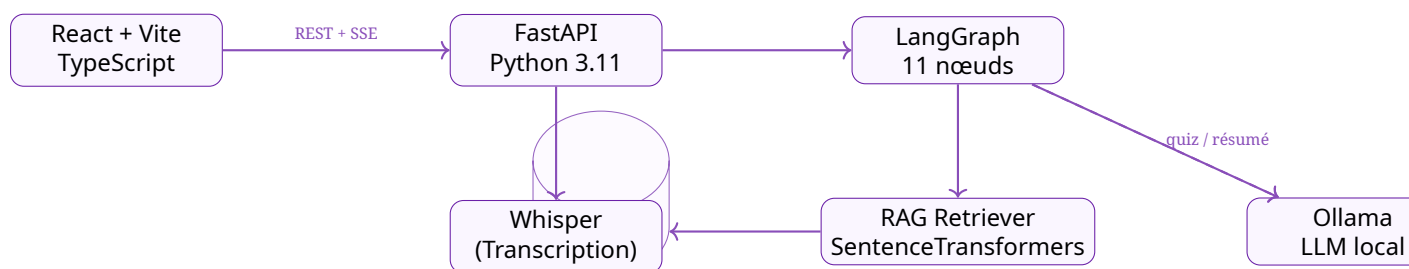


FIGURE 5 – Architecture logique du projet Mourchid.

4.2 Stack technologique

Couche	Technologies	Version
Frontend	React, TypeScript, Vite, Tailwind CSS, Framer Motion, Radix UI	React 18 / Vite 5
Backend API	Python, FastAPI, Uvicorn, SSE	Python 3.11+
Orchestration	LangGraph (graphe d'agents typé, traces persistées)	0.2+
RAG / Em-beddings	SentenceTransformers (multilingual-e5-base), Qdrant	—
LLM	Ollama (local, mistral, llama3.1...)	—
Transcription	Whisper via HuggingFace Transformers, PyTorch, FFmpeg	whisper-base
Packaging	uv (Python), npm (Node)	—
Déploiement	Docker Compose (Qdrant + backend + frontend)	—

TABLE 4 – Stack technologique open-source du projet.

4.3 Structure du dépôt

- **enset-assistant/**
 - backend/ FastAPI + LangGraph + RAG + MCP (uv)
 - app/main.py Point d'entrée API (routes + CORS)
 - app/api/ routes_chat, routes_quiz, routes_transcribe, routes_documents...
 - app/agents/ quiz_agent, summary_agent, citation_agent, verification_agent...
 - app/graph/ legal_workflow.py (LangGraph), state.py, nodes.py, conditions.py
 - app/rag/ embeddings, retriever, vector_store, chunker, indexer
 - app/llm/ local_llm.py (Ollama / llama.cpp), prompts.py
 - app/services/ chat_service, quiz_service, transcription_service, summary_service...
 - app/data/samples/ Corpus académique ENSET (Markdown)
 - pyproject.toml Dépendances Python (uv)
- **frontend/**
 - src/pages/ Chat.tsx, Quiz.tsx, Workflow.tsx, Dashboard.tsx, Summary.tsx
 - src/components/ chat/ (ChatComposer, VoiceVisualizer, MessageCard...), layout/ (Sidebar)
 - src/services/api.ts Appels REST (streamChat, generateQuiz, transcribeAudio...)
 - src/types/ index.ts, speech.d.ts
- **screens/**
 - assistant.png, transcription.png Captures interface chat et dictée vocale
 - QuizCntr.png, document.png Captures quiz et gestion documentaire
 - workFlow.png, dashboard.png, resumedocsEnset.png Autres écrans
 - docker-compose.yml Stack complète (Qdrant + backend + frontend)

4.4 Graphe LangGraph

Le cœur de l'application est un **graphe d'agents LangGraph** comportant **11 nœuds** interconnectés avec routage conditionnel :

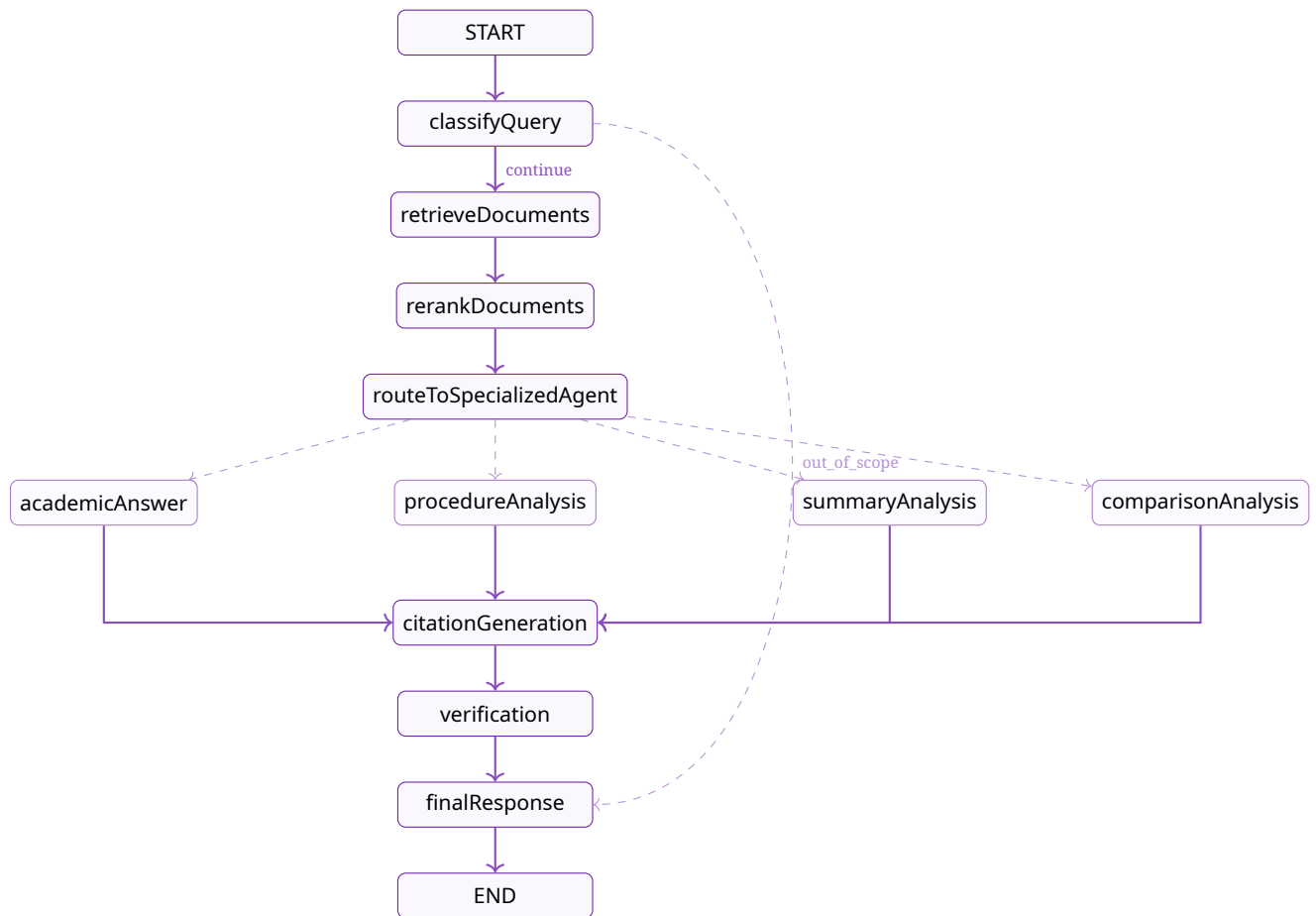


FIGURE 6 – Graphe LangGraph du pipeline multi-agent Mourchid (11 nœuds).

4.5 Endpoints REST principaux

Méthode	Route	Description
POST	/api/chat	Chat RAG agentic (réponse, citations, confiance, trace)
POST	/api/chat/stream	SSE : phases multi-agent + tokens streaming
POST	/api/quiz	Quiz QCM généré par le LLM (filière, difficulté)
POST	/api/transcribe	Transcription Whisper depuis un blob audio
POST	/api/summarize	Résumé structuré (points clés, obligations, risques)
POST	/api/documents/upload	Upload d'un fichier dans le corpus
POST	/api/documents/index	Indexation / réindexation du corpus
GET	/api/workflow/trace/{id}	Trace d'exécution persistée par conversation
GET	/api/health	État LLM, base vectorielle, modèle d'embedding

TABLE 5 – API REST FastAPI exposée au frontend React.

4.6 Composant de dictée vocale

```

1  async function startRecording() {
2    const media = await navigator.mediaDevices.getUserMedia({ audio: true });
3    setStream(media); // → VoiceVisualizer (Web Audio API)
4    const recorder = new MediaRecorder(media);
5    recorder.onstop = async () => {
6      const blob = new Blob(chunks, { type: recorder.mimeType });
7      const text = await transcribeAudio(blob); // POST /api/transcribe
8      setText(prev => prev ? `${prev} ${text}` : text);
9    };
10   recorder.start();
11 }

```

```

1  @lru_cache(maxsize=1)
2  def _get_pipeline():
3      return pipeline(
4          "automatic-speech-recognition",
5          model=settings.whisper_model, # openai/whisper-base par default
6          device=0 if torch.cuda.is_available() else -1,
7          chunk_length_s=30,
8      )
9
10 async def transcribe_audio(content: bytes, filename: str) -> str:
11     with tempfile.NamedTemporaryFile(suffix=".webm") as tmp:
12         tmp.write(content)
13         result = _get_pipeline()(tmp.name, generate_kwargs={
14             "language": "french", "task": "transcribe"
15         })
16     return result["text"].strip()

```

4.7 Génération de quiz par le LLM

```

1  async def generate_quiz(llm, filiere, count=5, difficulty="moyen"):
2      user = (f"Filiere:_{filiere}\nNombre:_{count}\n"
3             f"Difficulte:_{difficulty}\nGenere_un_quiz_original.")
4      # temperature=0.9 pour varier le quiz a chaque appel
5      raw = await llm.complete(QUIZ_SYSTEM, user, temperature=0.9, json_mode=
6          True)
7      data = safe_parse_json(raw)
8      return data.get("questions", [])

```

5. Réalisation

5.1 Interface assistant académique

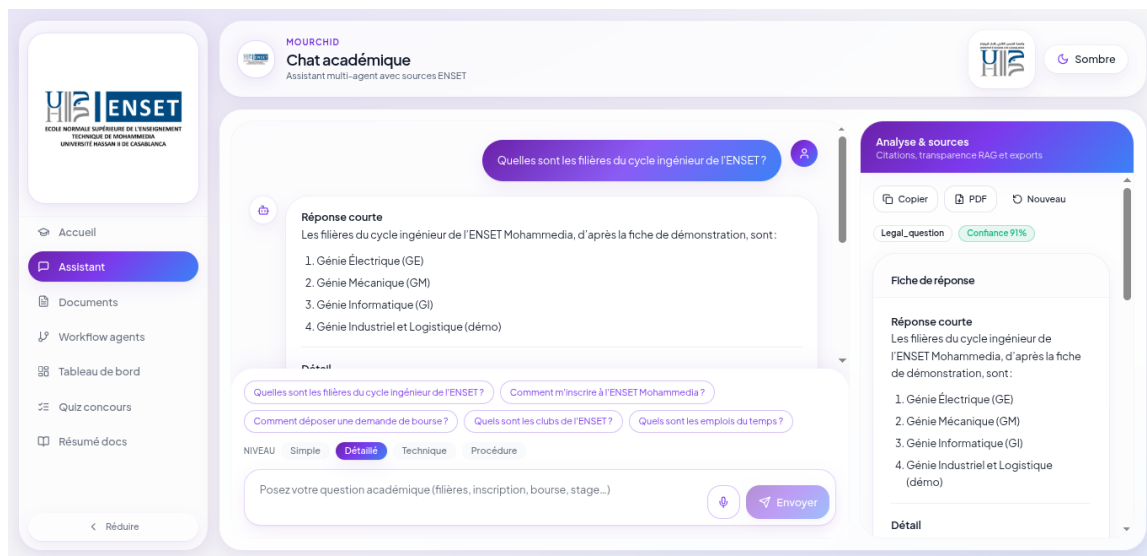


FIGURE 7 – Page Assistant — chat RAG en streaming SSE avec suggestions de questions, sélecteur de niveau (Simple / Détaillé / Technique / Procédure), questions prédéfinies et panneau latéral « Analyse & sources » affichant les citations, les extraits RAG et le score de confiance.

5.2 Dictée vocale avec barre de fréquence

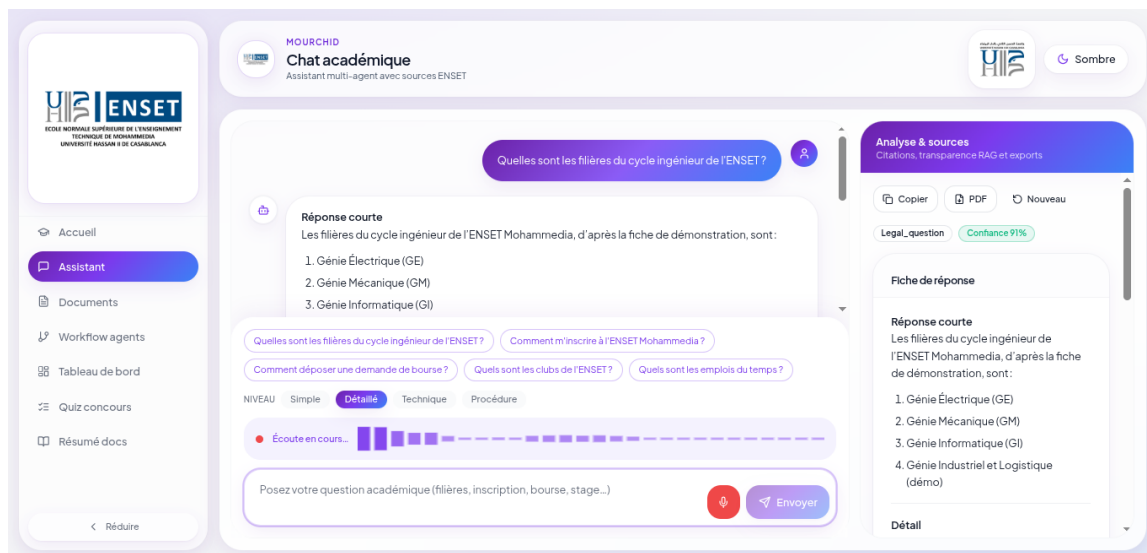


FIGURE 8 – Dictée vocale — enregistrement audio via MediaRecorder, visualisation en temps réel de l'amplitude par une barre de fréquence (Web Audio API + Canvas), puis transcription automatique via Whisper côté serveur et insertion du texte dans la zone de saisie.

5.3 Quiz concours généré par l'IA

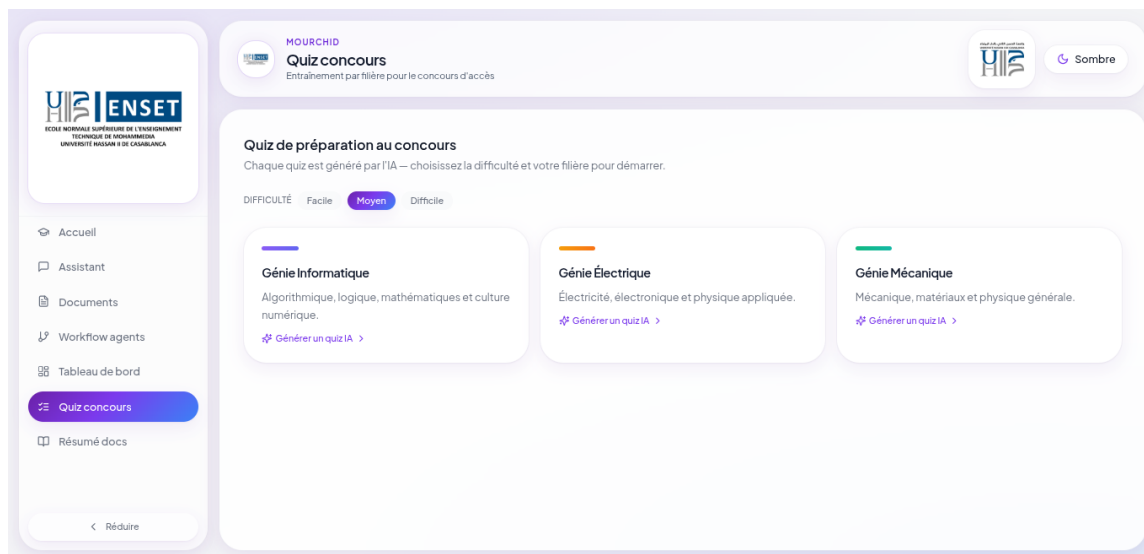


FIGURE 9 – Quiz concours — sélection de la filière (Génie Informatique, Électrique, Mécanique) et du niveau de difficulté ; chaque quiz est généré à la demande par le LLM (QCM avec 4 options, bonne réponse et explication pédagogique). L'interface affiche une barre de progression et une correction détaillée en fin de session.

5.4 Gestion documentaire

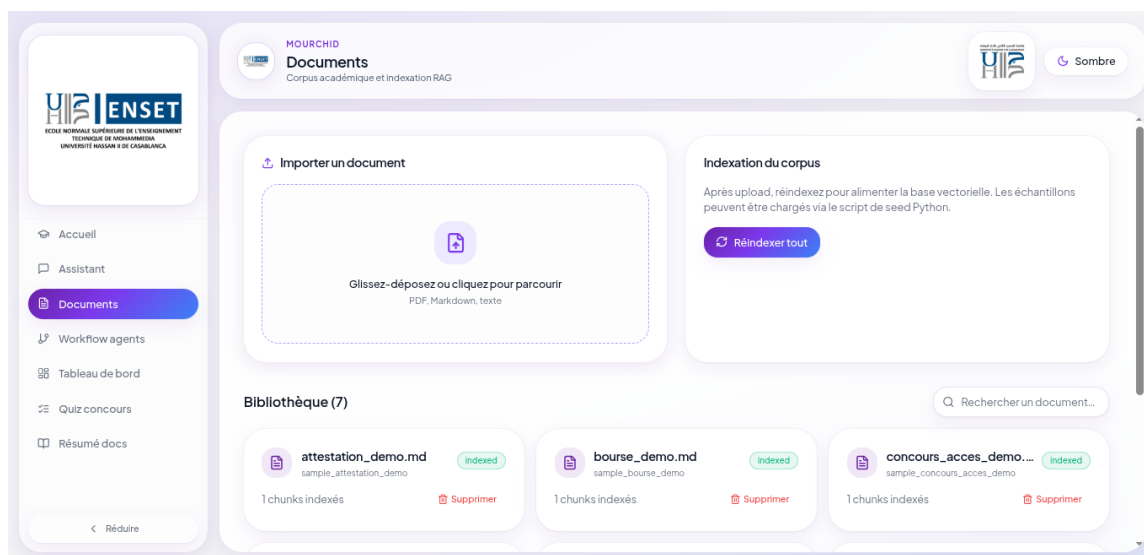


FIGURE 10 – Gestion du corpus documentaire — upload de fichiers PDF, Markdown ou texte, réindexation globale, statut d'indexation par fichier (nombre de chunks, identifiant). L'indexation alimente la base vectorielle Qdrant via les SentenceTransformers.

5.5 Visualisation du workflow multi-agent

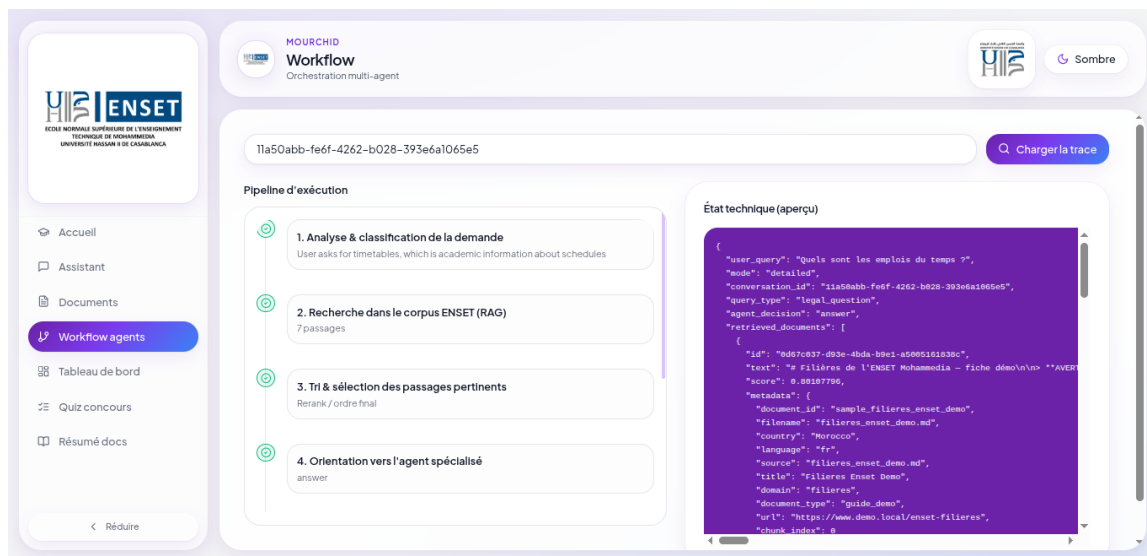


FIGURE 11 – Page Workflow — visualisation de la trace d'exécution du pipeline Lang-Graph par conversationId : chaque nœud (classifyQuery, retrieveDocuments, rerank-Documents, agent spécialisé, citationGeneration, verification, finalResponse) est affiché avec son détail et ses données de payload.

5.6 Tableau de bord

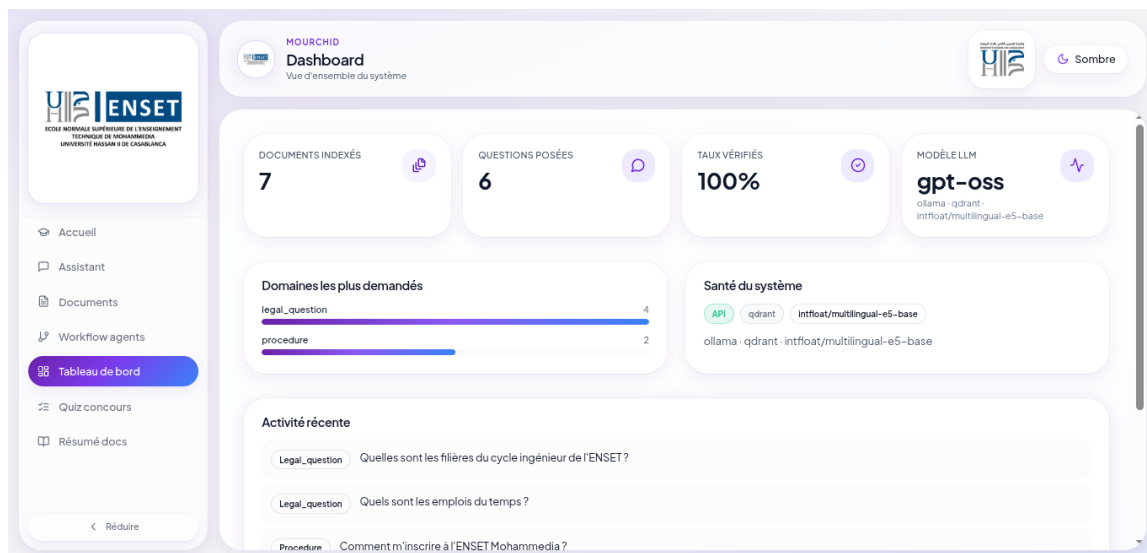


FIGURE 12 – Tableau de bord — statistiques d'utilisation (documents indexés, questions posées, taux de vérification réussi), modèles utilisés (LLM, embeddings, base vectorielle) et historique des dernières requêtes.

5.7 Résumé structuré de documents

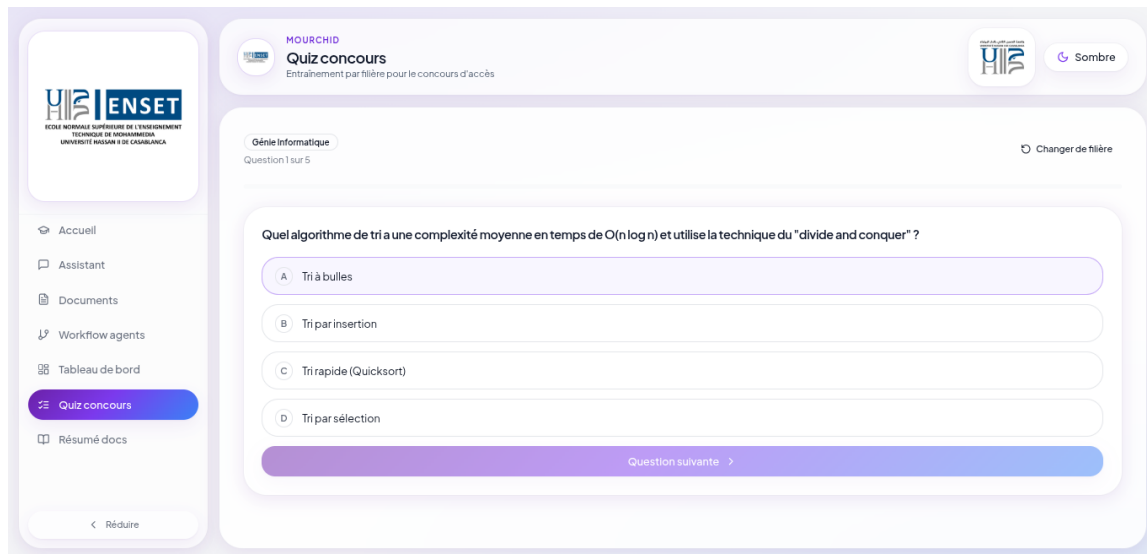


FIGURE 13 – Page Résumé — saisie d’un texte académique ou collage d’un document, puis synthèse structurée générée par le LLM : résumé principal, points clés, obligations, risques et articles de référence.

6. Installation et déploiement

Base vectorielle Qdrant

0

```
>_ sdia@enset : /enset-assistant
docker run -d -p 6333:6333 qdrant/qdrant
```

Backend Python (depuis backend/) — dépendances, corpus, serveur :

1

```
>_ sdia@enset : /enset-assistant/backend
uv sync --group dev
cp ../.env.example .env
uv run python -m app.scripts.seed_index
uv run uvicorn app.main:app --reload --port 8000
```

LLM Ollama (sur l’hôte) :

2

```
>_ sdia@enset : /enset-assistant
ollama serve
ollama pull mistral
```


Frontend React (depuis frontend/) :

3

```
>_ sdia@enset : /enset-assistant/frontend
npm install
npm run dev
# Interface disponible sur http://localhost:5173
```

**Checklist de validation**

- GET /api/health renvoie status: ok
- L'assistant répond avec citations et score de confiance
- La dictée vocale transcrit correctement (Whisper se charge au 1er appel)
- Le quiz est différent à chaque génération pour une même filière

**Premier lancement**

Au premier appel à la transcription vocale, le modèle Whisper (openai/whisper-base, ≈290 Mo) est téléchargé automatiquement depuis HuggingFace. Ce chargement initial est unique et mis en cache localement.

7. Conclusion

7.1 Résumé du projet

Le projet **Mourchid** démontre la faisabilité d'un assistant académique intelligent **local-first** et **open-source** répondant à un besoin réel des étudiants et candidats de l'ENSET Mohammedia. Il intègre avec succès :

- Un **pipeline RAG agentique** complet (Qdrant + SentenceTransformers + LangGraph) garantissant des réponses sourcées et auditable.
- Un **streaming SSE multi-phase** pour une expérience utilisateur fluide et transparente.
- Une **dictée vocale** robuste fonctionnant sur tous les navigateurs, grâce à la transcription Whisper côté serveur.
- Un **quiz concours dynamique** généré par l'IA, offrant un entraînement personnalisé par filière et par difficulté.
- Une architecture **100% locale** : aucune donnée n'est envoyée à des services tiers.

7.2 Défis rencontrés

Défi	Solution adoptée
Transcription vocale sur Linux	Abandon de l'API Web Speech (non fiable sur Chromium/Linux) au profit de Whisper côté serveur via MediaRecorder.
Variabilité des quiz LLM	Température élevée (0.9) pour diversifier les sorties + nettoyage robuste des questions mal formées.
Streaming bloquant le thread	Utilisation d' <code>asyncio.create_task</code> pour exécuter le graphe LangGraph en parallèle des événements SSE.
Hallucinations RAG	Nœud verification post-génération évaluant la fidélité aux extraits + message de repli si insuffisance documentaire.
Port occupé au redémarrage	Processus root lancé par <code>uv run</code> — résolu par <code>sudo fuser -k 8000/tcp</code> puis relance avec <code>--reload</code> .

TABLE 6 – Principaux défis et solutions apportées.

7.3 Recommandations et perspectives

- **Enrichissement du corpus** : indexer les guides officiels, règlements pédagogiques et emplois du temps réels de l'ENSET.
- **Support arabe** : les champs de métadonnées sont prévus ; une interface bilingue (FR/AR) est envisageable.
- **Human-in-the-loop renforcé** : activer le mécanisme interrupt de LangGraph avant `finalResponse` pour permettre à un modérateur de valider les réponses sensibles.
- **Whisper Large** : migrer vers `whisper-small` ou `whisper-large` pour une meilleure précision en français et en darija.
- **Déploiement hébergé** : conteneuriser via Docker Compose et déployer sur un serveur ENSET pour un accès centralisé sans installation locale.
- **Mobile-first** : adapter l'interface responsive pour une utilisation optimale sur smartphone (accès rapide à la dictée vocale en mobilité).

Mourchid illustre comment les technologies d'intelligence artificielle distribuée — orchestration multi-agent (LangGraph), RAG vectoriel (Qdrant), transcription neurale locale (Whisper) et génération LLM (Ollama) — peuvent être assemblées en une application web cohérente, ouverte et souveraine, au service de la communauté académique de l'ENSET Mohammedia.